

Example of GPIO Read and Write Operations

Example of GPIO Read and Write Operations

- 1. Expansion Pin Header Definition
- 2. GPIO Read And Write Operation Example
- 3. Environment Setup
- 4. How to Run
- 5. Expected Outcome

The development board features expansion pin headers, allowing users to perform peripheral expansions. Refer to this chapter for interface definitions.

1. Expansion Pin Header Definition

The RDKS600 has two 10-pin self-locking interfaces, one 12-pin self-locking interface, and one 14-pin self-locking interface, making it convenient for users to expand peripherals. The digital I/O is designed with a 1.8V logic level. The interface definitions are as follows:

UART	GPIO	I2C	SPI	ADC	CAN	LIN	PC1	GND	VDD_3V										
S600 RDK Pin 功能对照表																			
管脚功能1	管脚功能2	管脚功能3	管脚功能4	功能说明	管脚号	CVP功能名	管脚号	功能说明	管脚号	管脚功能5	管脚功能6	管脚功能7	管脚功能8	管脚功能9	管脚功能10	管脚功能11	管脚功能12	管脚功能13	管脚功能14
				管脚号	N/A	GND	1	GND	N/A										
				CAN5 差分信号	N/A	MCU_CAN5_H	2	MAIN_CAN5_H	N/A	CAN0 差分信号									
				CAN5 差分信号	N/A	MCU_CAN5_L	3	MAIN_CAN5_L	N/A	CAN0 差分信号									
				CAN6 差分信号	N/A	MCU_CAN6_H	4	MAIN_CAN6_H	N/A	CAN1 差分信号									
				CAN6 差分信号	N/A	MCU_CAN6_L	5	MAIN_CAN6_L	N/A	CAN1 差分信号									
				管脚号	N/A	GND	6	GND	N/A	管脚号									
				CAN7 差分信号	N/A	MCU_CAN7_H	7	MAIN_CAN7_H	N/A	CAN2 差分信号									
				CAN7 差分信号	N/A	MCU_CAN7_L	8	MAIN_CAN7_L	N/A	CAN2 差分信号									
				CAN8 差分信号	N/A	MCU_CAN8_H	9	MAIN_CAN8_H	N/A	CAN0 差分信号									
				CAN8 差分信号	N/A	MCU_CAN8_L	10	MAIN_CAN8_L	N/A	CAN0 差分信号									
				CAN9 差分信号	N/A	MCU_CAN9_H	11	MAIN_CAN9_H	N/A										
				CAN9 差分信号	N/A	MCU_CAN9_L	12	MAIN_CAN9_L	N/A										
				管脚号	N/A	GND	1												
GPIO_FUNC[13]		MCU_UART10_TX	LIN2_TXD	MCU_UART10 TX 信号	13	MCU_UART10_TX_V3	2	PC11_HCLK_V3	41	PC11_HCLK 信号	PC11_HCLK							GPIO_HSE[41]	
GPIO_FUNC[19]		MCU_UART10_RX	LIN2_RXD	MCU_UART10 RX 信号	19	MCU_UART10_RX_V3	3	PC11_BCLK_V3	42	PC11_BCLK 信号	PC11_BCLK	UART0_TXD	SPI1_CS0	GPIO_HSE[42]					
GPIO_FUNC[14]		MCU_UART11_TX	LIN3_TXD	MCU_UART11 TX 信号	14	MCU_UART11_TX_V3	4	PC11_FSYNC_V3	43	PC11_FSYNC 信号	PC11_FSYNC	UART0_RXD	SPI1_MOSI	GPIO_HSE[43]					
GPIO_FUNC[20]		MCU_UART11_RX	LIN3_RXD	MCU_UART11 RX 信号	20	MCU_UART11_RX_V3	5	PC11_DATA0_V3	44	PC11_DATA0 信号	PC11_DATA0	UART1_TXD	SPI1_MISO	GPIO_HSE[44]					
				管脚号	N/A	GND	6	PC11_DATA1_V3	45	PC11_DATA1 信号	PC11_DATA1	UART1_RXD	SPI1_SCLK	GPIO_HSE[45]					
				管脚号	N/A	GND	6	PC12_HCLK_V3	46	PC12_HCLK 信号	PC12_HCLK	UART1_TXD		GPIO_HSE[46]					
				管脚号	N/A	GND	6	PC12_BCLK_V3	48	PC12_BCLK 信号	PC12_BCLK	UART4_TXD	SPI2_CS0	GPIO_HSE[48]					
				管脚号	N/A	GND	6	PC12_FSYNC_V3	49	PC12_FSYNC 信号	PC12_FSYNC	UART4_RXD	SPI2_MOSI	GPIO_HSE[49]					
GPIO_HSE[52]	SPI1_CS0	UART6_TX	PC13_BCLK	UART6 TX 信号	52	MAIN_UART6_TX_V3	7	PC12_DATA0_V3	50	PC12_DATA0 信号	PC12_DATA0	UART5_TXD	SPI2_MISO	GPIO_HSE[50]					
GPIO_HSE[53]	SPI2_CS0	UART6_RX	PC13_FSYNC	UART6 RX 信号	53	MAIN_UART6_RX_V3	8	PC12_DATA1_V3	51	PC12_DATA1 信号	PC12_DATA1	UART5_RXD	SPI2_SCLK	GPIO_HSE[51]					
GPIO_HSE[54]	UART7_TXD		PC13_DATA0	UART7 TX 信号	54	MAIN_UART7_TX_V3	9	MAIN_I2C7_SDA_V3	31	I2C7_SDA 信号	I2C7_SDA	UART5_RXD	SPI3_CS0	GPIO_HSE[31]					
GPIO_HSE[55]	UART7_RXD		PC13_DATA1	UART7 RX 信号	55	MAIN_UART7_RX_V3	10	MAIN_I2C7_SCL_V3	30	I2C7_SCL 信号	I2C7_SCL	UART5_TXD	SPI2_CS0	GPIO_HSE[30]					
				管脚号	N/A	GND	1												
				管脚号	N/A	VDD_3V	14	5V 电源信号											

2. GPIO Read And Write Operation Example

In the development board's `/app/40pin_samples/` directory, various functional test codes for PIN pins are pre-installed, including GPIO input/output tests, I2C, SPI, UART, and other tests. All test programs are written in Python. Refer to other sections of this chapter for more details.

Take `/app/40pin_samples/button_led.py` as an example. This program configures pin 4 as input and pin 3 as output, controlling the output state of pin 3 based on the input state of pin 4.

3. Environment Setup

Use a jumper wire to connect pin 4 to 1.8V or GND to control its high/low logic level.

4. How to Run

Execute the `button_led.py` program to start the GPIO read/write program.

```
root@ubuntu:~# cd /app/40pin_samples/  
root@ubuntu:/app/40pin_samples# sudo python3 ./button_led.py
```

5. Expected Outcome

By controlling the high/low logic level of pin 4, you can change the output logic level of pin 3.

```
root@ubuntu:/app/40pin_samples# ./button_led.py  
Starting demo now! Press CTRL+C to exit  
Outputting 1 to Pin 3  
Outputting 0 to Pin 3  
Outputting 1 to Pin 3
```